underlying database ID of the entity you're editing. A simple attack would be to change that ID to gain access to a different entity, one that you wouldn't normally have access to. The user could try entering `/Recipes/Edit/121`. If that lets them edit or view a recipe that they shouldn't be able to, you have an insecure direct object reference vulnerability.

The solution to this problem is simple: you should have resource-based authorization in your endpoint handlers. If a user attempts to access an entity they're not allowed to access, they should get a permission-denied error. They shouldn't be able to bypass your authorization by typing a URL directly into the search bar of their browser.

In ASP.NET Core apps, this vulnerability typically arises when you attempt to restrict users by hiding elements from your UI, such as by hiding the Edit button. Instead, you should use resource-based authorization, as discussed in chapter 24.

WARNING You must always use resource-based authorization to restrict which entities a user can access. Hiding or disabling UI elements provides an improved user experience, but it isn't a security measure.

You can sidestep this vulnerability somewhat by avoiding integer IDs for your entities in the URLs, perhaps by using a pseudorandom globally unique identifier (GUID) such as `C2E296BA-7EA8-4195-9CA7-C323304CCD12` instead. This makes the process of guessing other entities harder, as you can't simply add 1 to an existing number, but it's masking the problem rather than fixing it. Nevertheless, using GUIDs can be useful when you want to have publicly accessible pages that don't require authentication but don't want their IDs to be easily discoverable.